

설비 고장을 예측하는 AI 데이터 분석

Pandas 데이터 선택·필터링

행·열 선택

C O N T E N T S

목차

01 열 선택의 기초 — 단일 열, 복수 열, 대괄호 차이

02 loc·iloc — 라벨과 위치, 슬라이싱 함정

01

열 선택의 기초

단일 열 · 복수 열 · 대괄호 한 겹과 두 겹의 차이

이 단원의 실데이터 — 주조 다이캐스팅

행.열 선택을 연습할 데이터가 어떤 설비에서 나왔는지 먼저 이해

공정 — 주조 다이캐스팅

용융 금속을 금형에 고압으로 주입해 부품을 성형하는 설비. 한 번의 주입(샷)마다 압력·두께·형체력을 기록한다.

핵심 변수와 물리적 의미

실린더압력·주조압력(주입 압력) · 사이클타임(1샷 소요 시간) · 비스킷두께(잔류 금속 두께) · 형체력(금형을 닫는 힘) · 품질등급(양품/주의/불량).

정상과 이상

압력·형체력이 정상 범위이고 비스킷두께가 적정하면 양품. 형체력 부족·비스킷 이상·사이클 지연이면 주의, 실제 결함이면 불량으로 기록된다.

왜 행.열 선택인가

수백 샷 로그에서 원하는 샷(행)과 지표(열)만 정확히 골라내야 품질을 빠르게 분석할 수 있다. loc(라벨)·iloc(위치)로 필요한 구간만 추출한다.

데이터에서 일부만 골라내는 이유

분석의 시작은 **필요한 부분만 정확히 꺼내기**

**전체가 아니라 필요한 부분만 정확히 꺼내는 것이
분석의 시작**

선택과 필터링은 앞으로 배울 모든 분석의 기초 동작

왜 손이 아니라 코드로 고르는가

엑셀 필터와 같은 일, 다만 두 가지가 결정적으로 다름

01 · 대용량 처리

수백만 줄도
한 줄로 순식간

엑셀은 수백만 줄에서 느려지지만 코드는 즉시 처리

데이터가 커져도 처리 속도 유지

02 · 재사용

내일 새 데이터에도
그대로 적용

한 번 만든 코드는 새 데이터에도 동일하게 동작

같은 작업의 자동 반복

DataFrame을 표로 떠올리기

엑셀 시트와 동일한 표 — 행 · 열 · 인덱스 세 개념

01 · 행

가로 한 줄

한 번의
측정 기록

한 시점에 잰 센서값들이 가로로 늘어선 줄

02 · 열

세로 한 칸

형체력·실린더압력 같은
측정 항목

같은 항목 값이 세로로 한 줄을 이룸

03 · 인덱스

행마다 붙는 이름표

기본은
0부터 시작

엑셀은 1부터, Pandas는 0부터 시작

DataFrame 구조 한눈에 보기

같은 항목은 세로 한 줄, 한 번의 측정은 가로 한 줄

열 (측정 항목) →

인덱스
행 이름표 ↓

idx	샷	품질등급	형체력	실린더압력	주조압력
0	1	양품	258	214	1037
1	2	양품	257	217	
2	3	주의	253	215	
3	4	양품	254	214	

← 행
하나의 측정 기록

세로로 고르면 형체력 항목 전체, 가로로 고르면 한 번의 측정 기록 전체

센서 하나가 열 하나

설비 데이터에서 열 하나는 대부분 센서 하나에 해당

형체력 열은 형체력 센서가 측정한 값
실린더압력 열은 실린더압력 센서가 측정한 값

필요한 열만 골라내면

데이터가 가벼워지고, 봐야 할 것에 집중 가능
센서 60개가 있어도 지금 필요한 두세 개에만 시선 고정

질문이 곧 열 선택으로 이어진다

무엇이 궁금한지를 정하면 어떤 열을 볼지가 따라옴

01 · 과열 점검

이 설비가 뜨거운가
→ **형체력 열**

과열 의심 → 형체력 센서값을 시계열로 확인

02 · 베어링 점검

떨림에 문제가 있는가
→ **실린더압력 열**

이상 실린더압력 의심 → 실린더압력 센서값으로 추세 확인

단일 열 선택

데이터프레임 뒤에 **대괄호**를 붙이고 열 이름을 **따옴표**로 감싸기

```
df['형체력']
```

위 식은 **형체력 열의 모든 값**을 세로로 출력

주의 — 따옴표 빠짐

따옴표를 빼면 파이썬이 **'형체력'**를 **변수 이름**으로 해석해 오류 발생
띄어쓰기·대소문자가 한 글자라도 다르면 **"그런 열은 없다"** 오류

단일 열 선택 — 코드

대괄호 + 따옴표로 열 하나 꺼내기

CODE

```
df = pd.read_csv('data/13_diecasting_small.csv')
형체력값 = df['형체력']
print(형체력값)
# 형체력 열 모든 값이 세로 출력
```

각 값 앞에 인덱스 번호가 함께 붙어 어느 측정의 형체력인지 추적 가능 · 큰따옴표도 결과 동일

Series는 한 줄짜리 데이터

열 하나를 꺼내면 표가 아닌 **1차원 데이터 Series**

값과 인덱스 이름표가 짝을 이룬 **똑똑한 목록**

출력하면 왼쪽에 0, 1, 2 인덱스, 오른쪽에 실제 값이 짝지어 나옴

0 63.8

1 64.5

2 71.2

Name: 형체력, dtype: float64

Series와 DataFrame 구분

쓸 수 있는 기능이 다르므로 `type()`으로 직접 확인

01 · 기능 차이

**Series에 바로
.mean() 적용 가능**

결과는 숫자 하나로 깔끔 · 표에서만 되는 작업도 따로 존재

02 · 확인 방법

`type(df['형체력'])`

찍어 보면 Series인지 DataFrame인지 즉시 확인 — 백 번 설명보다 확실

복수 열 선택

여러 열은 대괄호 안에 **리스트**로 묶어 선택 — 대괄호 두 겹

```
df[['형체력', '실린더압력']]
```

대괄호 두 겹의 의미

바깥 [] — 골라내기 신호

안쪽 [] — 리스트로 묶었다는 표시

적은 순서대로 결과 표의 열 순서 결정

자주 하는 실수

안쪽 대괄호 빠뜨림

대괄호 하나로 여러 이름을 쓰면 오류

여러 열은 **반드시 두 겹**

복수 열 선택 — 코드

열 이름들을 리스트로 묶어 여러 열 선택

CODE

```
두열 = df[['형체력', '실린더압력']]
print(두열.head())
# 형체력, 실린더압력 나란히 출력
순서변경 = df[['실린더압력', '형체력']]
```

결과는 DataFrame · 적은 순서가 그대로 열 순서 — 실린더압력을 먼저 적으면 실린더압력이 앞 열

대괄호 1개와 2개의 차이

같은 형체력 열도 **대괄호 겹수**에 따라 결과 형태가 달라짐

01 · 한 겹

`df[ '형체력' ]`

→ **Series**

한 줄짜리 1차원 데이터 — 바구니 없이 값만 꺼낸 것

02 · 두 겹

`df[[ '형체력' ]]`

→ **DataFrame**

열이 하나여도 표 형태 — 여러 열을 담을 수 있는 바구니

한 겹과 두 겹 결과 비교

두 겹은 항상 표 — 열이 하나든 여러 개든 일관된 규칙

구분	코드	결과 형태
한 겹	<code>df['형체력']</code>	Series
두 겹	<code>df[['형체력']]</code>	DataFrame
두 겹 여러 열	<code>df[['형체력','실린더압력']]</code>	DataFrame

두 겹은 항상 DataFrame — 예외 없이 적용되는 일관된 규칙

열 이름 확인

고르기 전에 `df.columns`로 정확한 열 이름 먼저 확인

```
print(df.columns)
```

여기 적힌 이름을 그대로 복사해 쓰면 오타 오류 방지

보이지 않는 공백·대소문자 차이도 미리 발견 가능
외부에서 받은 데이터일수록 예상과 다른 경우가 많음
시작 전 `columns`로 한 번 훑어보는 습관이 안전

새 데이터를 받으면 가장 먼저

모든 분석의 출발점 — 새 데이터를 받으면 세 가지를 묶어 기억

01 · head

데이터의 생김새

df.head()

앞부분 몇 줄을 빠르게 확인

데이터 첫인상 **미리보기**

02 · info

자료형과 결측치

df.info()

각 열의 자료형과 결측치 여부 한눈에 확인

데이터 **건강 점검**

03 · columns

정확한 열 이름

df.columns

공백·대소문자까지 데이터 그대로 노출

오타 오류의 **예방책**

이 세 가지를 습관화하면 뒤에서 생기는 오류가 크게 줄어듦

실습 1. 데이터 불러오기와 구조 확인하기

목표

설비 센서 csv를 불러와 크기와 열 이름 확인

단계

- read_csv로 설비 센서 파일 불러오기
- head로 앞부분, shape로 행.열 크기 확인
- columns로 열 이름 목록 확인

예상 결과

앞 5행, 크기 (30, 7), 열 이름 목록 출력

실습 — 결과 맞춰 보기

옆 사람과 출력 결과를 비교 — 환경을 맞춰 두면 다음 실습이 수월

체크포인트 — 같은 데이터를 제대로 불러왔는지

- ① `shape`가 (30, 7)로 같게 나오는지 확인
- ② `columns`의 열 이름이 동일한지 확인
- ③ 결과가 다르면 파일 경로·파일 이름 재확인

실습 2. 열 선택하기

목표

한 열(Series)과 여러 열(DataFrame)을 선택하고 바로 계산

단계

- 대괄호 한 겹으로 단일 열을 Series로 선택
- 대괄호 두 겹으로 복수 열을 DataFrame으로 선택
- 선택한 열에 mean으로 평균 계산

예상 결과

Series·DataFrame 형태와 형체력 평균 출력

실습 3. 공정 센서 열 골라내기

목표

주조 로그 공정 데이터에서 원하는 센서 열 선택

단계

- 주조 로그 파일 불러오기
- 한 센서 열을 Series로 선택
- 여러 feature 열을 DataFrame으로 선택해 형태 확인

예상 결과

한 열은 Series, 세 열은 (100, 3) 출력

02

loc과 iloc

라벨과 위치의 차이 · 슬라이싱 함정

행을 골라야 하는 상황

열은 어떤 항목, 행은 어떤 기록

특정 설비·특정 시점의 기록만 보려면
행 선택으로 시야를 좁힘

행 선택이 필요한 순간

현장의 질문은 대부분 **특정 대상·특정 시점**에 대한 것

01 · 특정 설비

3번 설비 기록만 떼어내기

지난주 문제 일으킨 설비를 전체에서 분리해 집중 분석

02 · 특정 구간

이상 의심 구간만 추출

이상 징후 의심 구간만 골라내 추세 확인

인덱스란 무엇인가

각 행에 붙은 이름표이자 행을 가리키는 주소

기본은 0부터 시작하는 자동 번호

표를 불러오면 맨 왼쪽에 0, 1, 2, 3 ... 번호가 자동 부여

첫 번째 행의 인덱스는 1이 아니라 0

1번 행을 보려고 1을 넣으면 → 두 번째 행이 나오는 함정

인덱스의 역할

인덱스는 단순한 번호가 아님 — 두 가지 결정적 역할

01 · 맞춤 기준

표 합치기·값 짝짓기는
같은 인덱스끼리

인덱스가 어긋나면 데이터가 엉뚱하게 짝지어짐

02 · 이름 변경

품질등급·날짜를
인덱스로 지정

의미 있는 이름으로 호출 가능 — 인덱스가 꼭 숫자일 필요는 없음

라벨과 위치 번호의 차이

오늘 모듈에서 **가장 중요한 개념** — 라벨이나 위치나

01 · 라벨

그 행에 붙은
이름표 = 인덱스 값

인덱스 값 자체 — 0, 17, 양품처럼 데이터가 들고 있는 라벨

02 · 위치 번호

맨 위에서부터
몇 번째인지

표 안에서의 순서 — 항상 0부터 시작

라벨과 위치는 어긋날 수 있다

구분 못 하면 엉뚱한 행을 분석하게 됨

01 · 아파트 비유

101호 = 이름표
1층 둘째 = 위치

보통은 일치하지만 리모델링으로 호수 재부여 시 어긋남

02 · 정렬 후

맨 위 행 인덱스가
17처럼 바뀜

형체력순 정렬 시 라벨과 위치가 분리 — 도구도 따로 필요

라벨과 위치가 갈라지는 순간

정렬 전에는 라벨 = 위치, 정렬 후에는 둘이 갈라짐

정렬 전 — 라벨 = 위치

라벨	위치	형체력
0	0	63.8
1	1	64.5
2	2	71.2
3	3	69.7



정렬 후 — 라벨 ≠ 위치

라벨	위치	형체력 ↓
		71.2
		69.7
		64.5
		63.8

같은 행을 라벨로는 3, 위치로는 0번째로 부름 — 도구가 둘로 나뉨

loc과 iloc의 핵심 구분

오늘 모듈의 **심장** — 이 두 글자 차이가 전부

`DF.LOC[ ]`

이름표
라벨로 선택

인덱스가 품질등급·날짜 같은 글자여도 사용 가능

`DF.ILOC[ ]`

순서
위치 번호로 선택

i = integer — 0부터 시작하는 정수만 받음

loc과 iloc 비교표

네 가지 비교 항목 — 기준 · 받는 값 · 글자 인덱스 · 조건 필터

구분	loc	iloc
기준	라벨 (이름표)	위치 (순서)
받는 값	인덱스 값	0부터 시작하는 정수
글자 인덱스	가능	불가
조건 필터	가능	불가

조건 필터(예: 형체력 > 80) — 다음 시간에서 loc과 함께 활용

loc·iloc 혼동 주의

열은 대괄호로, 행은 loc 또는 iloc으로

01 · 점과 대괄호

`df.loc[0]` ✓

`df.loc(0)` ✗

점 뒤는 대괄호 — 함수처럼 소괄호 쓰면 오류

02 · 열 선택과 구분

`df['형체력']` · 열

`df.loc[0]` · 행

이름만 대괄호 → 열 · loc/iloc이 앞에 → 행

loc 단일 행 선택

df.loc 뒤 대괄호에 **인덱스 라벨** 작성

```
df.loc[0]
```

이름표가 **0인 행** 전체를 선택

중요 — 여기서 0의 의미

여기서 0은 위치(첫 번째)가 아니라 이름표

지금은 인덱스가 순서대로라 결과가 같아 보이지만 의미는 어디까지나 이름표

loc 단일 행의 특징

행 하나를 90도 돌려 세운 모습 — 그 시점의 모든 센서값을 한눈에

01 · 결과는 Series

한 행의 여러 값이
세로로 늘어선 형태

그 측정 시점에 모든 센서가 보인 값을 한눈에 확인

02 · 글자 라벨 가능

`df.loc['양품']`

인덱스를 품질등급·날짜로 지정하면 이름으로 바로 호출

loc 행 범위 선택

콜론으로 범위 — loc은 끝 라벨까지 포함

```
df.loc[0:4]
```

0, 1, 2, 3, 4 — 다섯 줄

loc만의 매우 중요한 특징

loc은 끝에 적은 라벨까지 포함 — 문자열 슬라이싱과 정반대
잠시 후 배울 iloc은 끝을 포함하지 않아 결과가 다름

loc 범위 활용

구간 좁히기와 생략 — 끝 포함 규칙은 그대로

01 · 구간 좁히기

`df.loc[10:20]`

이상 의심 구간이 10~20번 → 그 구간만 떼어내 집중 분석

02 · 생략 가능

`df.loc[:5]` · 처음~5

`df.loc[10:]` · 10~끝

시작·끝 생략 가능 — 끝 라벨 포함 규칙 동일

loc 행과 열 동시 선택

loc의 진짜 힘 — 사각형 영역만 정확히 추출

```
df.loc[행, 열]
```

심표 앞은 행, 심표 뒤는 열 — 행 먼저, 열 나중

외우는 법 — 줄, 즉 행이 먼저

표 읽을 때도 "몇 번째 줄의 어느 칸" — 순서 헷갈리면 엉뚱한 데이터

loc 행과 열 — 코드

심표 앞은 행, 뒤는 열 — 콜론 하나만 적으면 전체

CODE

```
부분 = df.loc[0:2, ['품질등급', '형체력']]  
print(부분)  
전체행 = df.loc[:, ['형체력', '실린더압력']]
```

loc은 끝 라벨 포함 — 0:2는 세 줄 · 행 자리에 콜론 하나면 모든 행

iloc 위치 기반 선택

인덱스와 무관 — 위에서부터 세는 순서

```
df.iloc[0]
```

인덱스가 무엇이든 맨 위 첫 번째 행

iloc의 두 가지 제약

- ① 정수만 받음 — '양품' 같은 글자는 오류 · i = integer
- ② 조건 필터링 불가 — 순서 번호와 그 범위만

iloc 활용

상위 N개·음수 위치 — 순서로 세기에 가능한 기능들

01 · 상위 N개

```
df.iloc[0:5]
```

정렬 후 맨 위 인덱스를 몰라도 상위 5개를 정확히 추출

02 · 음수 위치

```
df.iloc[-1] · 마지막
```

```
df.iloc[-3:] · 마지막 3줄
```

가장 최근 측정 기록을 볼 때 편리 — 순서로 세기에 가능

iloc 행과 열 동시 선택

loc과 동일한 쉽표 구조 — 이름 대신 순서 번호

```
df.iloc[행번호 , 열번호]
```

열도 0부터 — 0번 샷, 1번 품질등급, 2번 형체력, ...

iloc 슬라이싱 — 끝 미포함

열 위치 1:3 → 1번과 2번(품질등급·형체력)

헛갈리면 columns로 열 순서 미리 확인

iloc 행과 열 — 코드

행도 열도 순서 번호로 지정

CODE

```
print(df.iloc[0:2, 1:3])  
# 0~1번 행, 1~2번 열  
# = 실린더압력, 주조압력
```

같은 영역을 loc은 이름으로, iloc은 번호로 짚는 두 가지 길

loc과 iloc 슬라이싱 끝 포함 차이

오늘 수업에서 **가장 실수가 잦은 부분** — 집중

01 · loc

```
df.loc[0:2]
```

0, 1, 2 → **세 줄**

끝 라벨까지 포함 — 부른 이름은 다 옴

02 · iloc

```
df.iloc[0:2]
```

0, 1 → **두 줄**

끝 미포함 — 파이썬 기본 슬라이싱 규칙

슬라이싱 줄 수 비교

loc은 항상 한 줄 더 많음 — 끝을 포함하니까

코드	가져오는 행	줄 수
df.loc[0:2]	0, 1, 2번	3줄
df.iloc[0:2]	0, 1번	2줄
df.loc[0:4]	0 ~ 4번	5줄
df.iloc[0:4]	0 ~ 3번	4줄

외우는 문장 — loc은 부른 이름까지 다 옴, iloc은 끝 직전까지만 옴

슬라이싱 차이가 부르는 실수

의도를 분명히 — 개수냐 라벨이냐

01 · 11개 함정

```
df.loc[0:10]
```

10개가 아니라 11개

더 무서운 건 오류가 안 남 — 코드는 멀쩡히 돌고 결과도 그럴듯

02 · 의도 구분

정확히 N개 →

iloc

이름표 범위 →

loc

의도에 맞는 도구 선택

실습 4. loc와 iloc로 행 선택하기

목표

라벨 기준 loc와 번호 기준 iloc로 행 선택, 범위 차이 확인

단계

- loc로 라벨 기준 단일 행 선택
- iloc로 번호 기준 단일 행 선택
- 범위 선택으로 loc 끝 포함·iloc 끝 제외 차이 확인

예상 결과

품질등급 값과 범위 줄 수 3·2 출력

실습 5. loc·iloc로 행·열 동시 선택하기

목표

행과 열을 동시에 지정해 원하는 부분만 추출

단계

- loc로 행 범위와 열 이름을 함께 지정
- 다른 행 범위에서 세 열 선택
- iloc 음수 인덱스로 마지막 행 선택

예상 결과

(5, 2)·(5, 3)·마지막 3행 출력

실습 6. 특정 구간 추출 종합

목표

열 선택·loc·iloc를 결합해 특정 구간을 추출하는 종합

단계

- 여러 feature 열을 선택한 뒤 iloc로 앞 구간 추출
- loc 라벨 범위로 두 열 구간 추출
- iloc 위치 범위로 앞쪽 열 구간 추출

예상 결과

(10, 5)·(11, 2)·(10, 6) 출력

행.열 선택 핵심 정리

오늘 단 하나만 가져간다면 — 이 두 문장

핵심 01

이름표로 고를 땐 **loc**, 순서로 고를 땐 **iloc**

핵심 02

loc은 끝까지 포함, **iloc**은 끝 직전까지

열 선택 정리

열을 고를 때는 **대괄호** — 두 가지 핵심 습관

01 · 한 겹과 두 겹

한 겹 → **Series**

두 겹 → **DataFrame**

한 겹은 이름 하나, 두 겹은 이름 목록 — 형태가 갈림

02 · 이름 확인

df.columns

고르기 전 columns로 정확한 열 이름 점검 — 오타 오류의 예방책

행 선택 정리

선택 능력이 다음 시간 조건 필터링의 바탕

01 · loc

라벨로 선택

끝 포함 · 조건 필터 가능

이름표로 부르니까 부른 이름까지 다 옴 — 조건 필터의 무대

02 · iloc

위치로 선택

끝 제외 · 정수만

파이썬 기본 슬라이싱 규칙 — 정확히 N개가 필요할 때